Welcome, Priyanka! I'm Claude.

Bring me anything—a tough problem, a half-formed idea, something you need to write. We'll figure it out together.

Where do you want to start?

Frontend: React
Backend API: Java Spring Boot
Database: PostgreSQL
File storage: on-prem file server
Auth: internal login
Reporting: built-in dashboard + export to Excel/PDF
Reports: JasperReports / server-side PDF + Excel export
That stack is stable and audit-friendly.
How to set up the environments and the folder structure for this tech stack. I have to download and transfer the m2 folder and node modules and everything else required for development in a system without internet and the development will happen without internet. So give step by step instructions to achieve this without much glitch. I have jdk 25 in my system.

Validated Java 25 compatibility and architected offline-first deployment strategy

Good stack choice for an audit-heavy environment — all mature, well-documented components. Since you're air-gapping this, the real risk isn't the code, it's silent version/architecture mismatches that only surface once you're already offline. Let me walk through this properly.

0. One compatibility flag before you start

You're on JDK 25. Confirmed current state: **Spring Boot 3.5.5+ and Spring Boot 4.0 are Java 25-ready**, so you're fine on the framework side. The one thing to actually test *before* you go offline is **JasperReports** — it's an older library and some versions lag on newer JDKs (reflection/module-access issues show up at runtime, not compile time). So:

- Pick JasperReports 7.x (latest) — better chance of JDK 25 compatibility.
- Actually generate a test report (PDF + Excel export) on JDK 25 while you still have internet, before finalizing what goes into your offline bundle. If it breaks, you have two clean fallbacks: install JDK 21 (LTS) alongside JDK 25 and target the backend at 21 via Maven toolchains, or find the correct compatible Jasper version — either is a 5-minute fix online and a nightmare offline.

1. Recommended folder structure

```
project-root/
├── frontend/                 # React app
│   ├── src/
│   ├── public/
│   ├── package.json
│   ├── package-lock.json
│   └── .npmrc                # points to local offline registry/cache
├── backend/                  # Spring Boot app
│   ├── src/main/java/...
│   ├── src/main/resources/
│   │   └── reports/          # .jrxml Jasper templates
│   ├── pom.xml
│   └── .mvn/                 # maven wrapper (mvnw)
├── db/
│   ├── migrations/           # Flyway/Liquibase scripts
│   └── seed/
├── docs/
│   └── offline-setup/        # this guide + checksums, versions.txt
└── offline-bundle/           # NOT committed to source control — see below
    ├── jdk/
    ├── maven-repo/            # mirrored .m2
    ├── node/                  # node binaries
    ├── npm-cache/              # or node_modules snapshots
    ├── postgres-installer/
    └── tools/                 # IDE installers, Jasper Studio, etc.
```

Keep `offline-bundle/` **outside** git — it's large binary transfer material, not source. Ship it separately (USB/DVD/internal file share) alongside a `versions.txt` manifest listing exact versions of everything, so six months from now someone can reproduce the exact environment.

2. On an internet-connected staging machine — build the transfer bundle

Do all of this on one throwaway machine with the *same OS/architecture* as your offline dev machines (Windows vs Linux matters for binaries).

a) JDK

You already have JDK 25 locally — just zip up the JDK install directory itself (e.g. Temurin 25 distribution) so you can drop it onto machines that don't have it. No special export step.

25 distribution) so you can drop it onto machines that don't have it. No special export step needed, it's just files.

b) Maven `.m2` repo (the important one)

Don't just zip your existing `~/.m2` — it'll have partial/corrupt entries. Do a clean pull instead:

```bash
# From backend/ with pom.xml fully fleshed out (all deps declared)
mvn dependency:go-offline -Dmaven.repo.local=/path/to/clean-m2
```

This forces Maven to resolve and download **every** dependency (including plugins, their dependencies, and transitive deps) into a fresh local repo folder. Also explicitly run a full build once against that repo to catch anything `go-offline` misses (some plugins fetch things only during actual `package` / `test` phases):

```bash
mvn clean package -Dmaven.repo.local=/path/to/clean-m2
```

Then zip `/path/to/clean-m2` — that's your transferable `.m2`.

Critical: also copy your `pom.xml`'s exact plugin versions (compiler plugin, surefire, spring-boot-maven-plugin, jasperreports-maven-plugin if you use it) — pin every version explicitly. Don't rely on version ranges or "latest" — offline, Maven can't resolve a moving target.

c) Node / npm side

Two viable approaches — pick one:

Option 1 (simplest, works well for a single project): ship `node_modules` directly

```bash
cd frontend
npm ci   # clean install from package-lock.json
```

Then zip `node_modules/` **built on the same OS** as the offline target (native modules like `esbuild`, `sharp`, `node-sass` compile OS-specific binaries — a `node_modules` built on Windows won't work on Linux and vice versa). This is fine for one project but doesn't scale if you'll add more JS projects later.

Option 2 (better long-term): local npm cache + offline registry

```bash
npm ci
npm cache verify
```

Then zip your npm cache folder (find it via `npm config get cache`, typically `~/.npm`). On the offline machine, point npm at this cache and run `npm ci --offline`. This is more portable across projects but a bit more setup.

For a single stable stack like yours, **Option 1 is simpler and less to go wrong** — go with that unless you expect to spin up more frontend projects later.

Also zip the **Node.js binary/installer itself** — the offline machine needs Node runtime, not just node_modules.

d) PostgreSQL

Download the full installer (Windows `.exe`/Linux `.deb`/`.rpm` or the binary tarball) for your target OS — not just the client. If you'll containerize, also pull the Docker image and `docker save` it to a `.tar` (only if Docker itself is allowed offline, which needs its own bundling — flag this if relevant, it's a common gotcha since Docker Desktop wants to phone home for updates).

e) IDE and misc tools

- IDE installer (IntelliJ/VSCode) + any offline plugin bundles (Lombok plugin, Spring plugin, etc. — these are also downloadable as offline plugin zips from the IDE's plugin marketplace).
- Jasper Studio if you're hand-designing `.jrxml` templates visually (bundle its installer too).
- Git itself, if not already on target machines.
- Any Chrome/browser dev tools extensions you rely on.

3. Transfer

Checksum everything before and after transfer (`sha256sum`) — corrupted zips of a 500MB `.m2` are the single most common cause of "works on staging, fails offline" support tickets. Write the checksums into `docs/offline-setup/versions.txt` alongside exact version numbers of every component.

4. On the offline machine — set up

1. **Install JDK** — unzip, set `JAVA_HOME`, add to `PATH`.
2. **Install Node** — unzip/install binary, confirm `node -v` / `npm -v`.
3. **Configure Maven to use local-only repo**: In `~/.m2/settings.xml`:

```xml
<settings>
  <mirrors>
    <mirror>
      <id>offline-repo</id>
      <mirrorOf>*</mirrorOf>
      <url>file:///path/to/clean-m2</url>
    </mirror>
  </mirrors>
  <offline>true</offline>
</settings>
```

Then just `mvn clean package` from `backend/` — it should resolve entirely from the local mirror with zero network calls.

4. **Frontend**: copy the pre-built `node_modules` next to `package.json`, then just run `npm run start` / `build` directly — skip `npm install` entirely since node_modules is already there (running `npm install` offline without a registry will just fail/hang, so don't trigger it by accident — e.g. don't delete `node_modules` before testing).
5. **PostgreSQL**: run the installer, create the DB, restore schema via your migration scripts.
6. **IDE**: install from the offline installer, then install plugins from the downloaded plugin zips via "Install Plugin from Disk" rather than the marketplace.
7. **Validate immediately**, in this order, before declaring the machine ready:
   - `mvn clean package` succeeds with zero network hits (temporarily disconnect network to be sure).
   - `npm run build` succeeds.
   - Generate one JasperReports PDF and one Excel export end-to-end.
   - Connect backend to PostgreSQL and run one migration.

That last validation step matters most — it's much cheaper to discover a JasperReports/JDK 25 mismatch or a missing transitive Maven dependency *now*, while you can still go fetch one missing jar, than after the machine is fully sealed off.

If you want, I can help draft the actual `pom.xml` dependency block (Spring Boot + JasperReports + PostgreSQL driver) with pinned versions, or a shell/PowerShell script that automates steps 2–4 on the staging machine — just let me know which OS your dev machines run.

Does server side pdf generation is better or client side ?

Tailored recommendation by weighing context-specific constraints

For your setup specifically — **server-side is the better fit**, and here's why, plus the general tradeoffs so you can sanity-check that against your actual requirements.

Why server-side wins for your case

- **Audit-friendliness**: You said this stack needs to be audit-friendly. Server-side generation means the PDF is built from the same data the DB actually has, using a controlled template (JasperReports `.jrxml`), with no dependence on what browser/JS version rendered it. That's reproducible and defensible in an audit — client-side rendering (e.g. via `jsPDF`, `html2canvas`) can subtly differ across browsers/devices, which is a bad look when someone asks "why does this PDF from last month look slightly different."
- **Consistency of formatting**: JasperReports gives you precise control over pagination, headers/footers, fonts, page breaks — things that are genuinely painful to get pixel-perfect with client-side HTML-to-PDF tools, especially for tabular financial/report-style documents.
- **Security/data exposure**: Server-side means sensitive data never has to be shipped to the browser and reconstructed there — it's rendered once and only the final PDF crosses the wire. For an internal-auth, on-prem system, that's a meaningfully smaller attack surface.
- **You already have the toolchain**: JasperReports is a server-side tool. Fighting your own architecture to also do client-side generation would be duplicated effort for no real gain here.
- **Offline environment**: this actually matters a lot given your air-gapped setup — server-side rendering has zero dependency on CDN-hosted fonts/JS libraries that client-side PDF libraries often reach for. One less thing to bundle and validate offline.

When client-side would actually be the better call (for contrast)

- **Instant, throwaway previews** — e.g. a user wants to see roughly what an invoice will look like before submitting, and it doesn't need to be archived or audited. Round-tripping to the server for every tweak adds latency.
- **Reducing server load** — if you expect very high volume of report generation and want to offload CPU work to the client's browser instead of your Spring Boot servers.
- **No backend involvement needed** — e.g. exporting a chart or table that's purely client-rendered data with no server template.

None of those really apply to your reporting use case (dashboard reports, Excel/PDF

exports meant to be official records) — so stick with server-side JasperReports as your primary path. If you want, a reasonable middle ground is: server-side for the "real" exports/reports that get archived or audited, and a lightweight client-side "quick preview" (even just rendering the same data as an HTML view before the user clicks "Export") for UX responsiveness — but the artifact of record stays server-generated.

Can we do both for the same pdf?

Weighed hybrid PDF generation approaches and implementation tradeoffs

Yes — but it's worth being precise about *what* "both" means, because there are a few different patterns and only some of them make sense for an audit-friendly system like yours.

Pattern 1: Client-side preview, server-side is the real PDF (recommended)

This is the sane hybrid: the client renders an HTML/CSS preview that *visually approximates* the report using the same data, and only when the user confirms/exports does the request go to Spring Boot + JasperReports to generate the actual file.

- The client preview is **not** a PDF at all — just a styled HTML view. This avoids the drift problem entirely (see below).
- The server-generated PDF is the only artifact that ever gets saved/archived/audited.
- Good for UX responsiveness without compromising your "single source of truth" requirement.

Pattern 2: Generate the same PDF via both paths (generally not recommended)

If you mean literally: JasperReports produces a PDF server-side, *and* something like `jsPDF` / `html2canvas` also produces a PDF client-side from the same data — technically possible, but creates real problems for you specifically:

- **Drift risk**: two independent rendering engines (Jasper's layout engine vs. a browser's) will never pixel-match. Fonts, pagination, and spacing will differ subtly. For an audit trail, having two "versions" of the same report that don't look identical is a liability, not a convenience.
- **Double maintenance**: every template change means updating both the `.jrxml` and the client-side rendering logic, and keeping them in sync forever.
- **Which one is authoritative?** If a client-generated PDF and a server-generated PDF for the "same" report ever disagree, you've created an audit question rather than avoided one.

If you have a genuine reason for this (e.g. offline-capable client that must generate a PDF without hitting the server at all in some scenario), it's doable — just treat the client-side one explicitly as a *local convenience copy*, never store it as the official record, and maybe watermark it "unofficial/local copy" so nobody confuses the two later.

Pattern 3: Same PDF, viewed both places

If "both" actually means "generate it once server-side, then let the client view/manipulate that exact PDF" (e.g. render it in-browser with `pdf.js` for a print preview before download) — that's just Pattern 1's cousin and is completely safe: one generation, one file, multiple places it's displayed.

My suggestion: go with Pattern 1. It gives you the responsiveness benefit people usually want from "client-side PDF" without any of the audit/consistency risk. Want me to sketch out what that flow would look like — e.g. a React component that renders the preview from the same DTO/JSON your Spring Boot endpoint would feed to Jasper, so the preview and the final PDF are guaranteed to reflect the same data shape?

What all dependencies will be needed for springboot and react.

Searched the web

Here's the full list, organized by concern. I'll flag the ones that need careful version-pinning given your air-gapped setup and JasperReports/Jakarta compatibility.

Spring Boot (`pom.xml`)

Core

- `spring-boot-starter-web` — REST API layer
- `spring-boot-starter-data-jpa` — PostgreSQL access via Hibernate
- `spring-boot-starter-validation` — request/DTO validation
- `spring-boot-starter-security` — since you're doing internal auth (session/JWT-based)
- `spring-boot-starter-actuator` — health checks, useful even offline for ops visibility
- `spring-boot-devtools` (dev scope only) — hot reload during development

Database

- `org.postgresql:postgresql` — JDBC driver (pin the exact version; don't use a range)
- `flyway-core` or `liquibase-core` — schema migrations (pick one; Flyway is simpler, Liquibase more flexible for complex audit trails)

Reporting — this is the one to get right

- `net.sf.jasperreports:jasperreports` — **use 7.0.x, not 6.x**. Important: JasperReports 7.0+ is the first version with Jakarta EE namespace support (`jakarta.*` instead of `javax.*`), which is what Spring Boot 3.x requires. If you accidentally pull a 6.x version, you'll get classpath conflicts that are painful to debug, worse to debug offline.
- `net.sf.jasperreports:jasperreports-fonts` — bundles common fonts so PDF rendering doesn't depend on OS fonts being present (matters a lot for reproducibility across machines)
- `net.sf.jasperreports:jasperreports-functions` — if your templates use JR's built-in functions
- For Excel export specifically, JasperReports' own `JRXlsxExporter` is included in the core jar — no separate dependency needed unless you want raw Apache POI control instead, in which case add `org.apache.poi:poi-ooxml` directly.

File handling / on-prem storage

- `commons-io` — file utilities
- Nothing special needed for "on-prem file server" unless it's SMB/NFS-mounted (then it's just filesystem I/O, no extra dependency) or you're integrating over an API/protocol like FTP/SFTP — if so, tell me which and I'll add the right client library.

Utility

- `lombok` — reduces boilerplate (confirm your Lombok version is JDK 25-compatible — 1.18.42+ noted as fine in the migration notes for JDK 25)
- `mapstruct` — DTO ↔ entity mapping, if you want it (also double-check its JDK 25 compatibility before you lock it in)

Build plugins (pin exact versions, don't rely on parent BOM defaults for these)

- `spring-boot-maven-plugin`
- `maven-compiler-plugin` — set `<release>` explicitly (17, 21, or 25 depending on what you land on after your Jasper compatibility test)
- `jasperreports-maven-plugin` — worth noting: as of the 7.0.6 release, JasperReports ships an **official** Maven plugin (`net.sf.jasperreports:jasperreports-maven-plugin`) for compiling `.jrxml` → `.jasper` at build time instead of at runtime. Worth using — it means your reports get pre-compiled during the build and you're not paying compile cost (or risking a compile failure) at runtime in production.

React (`package.json`)

Core

- `react`, `react-dom`
- `react-router-dom` — routing
- `axios` (or native `fetch` wrapped in a small client) — API calls to Spring Boot
- Build tool: `vite` (recommended over CRA — CRA is effectively unmaintained now) or if you already have a CRA/webpack setup, that's fine too, just know Vite will be far less painful for the offline dependency story since it has a smaller, more modern dependency tree.

State/forms (pick based on your app's complexity — don't over-add)

- `@tanstack/react-query` — server state/caching, pairs nicely with Spring Boot APIs
- `react-hook-form` + `zod` (or `yup`) — form handling + validation
- Redux/Zustand only if you have genuinely complex client-side state; skip otherwise

UI

- A component library — `Ant Design` or `MUI` fit "internal enterprise dashboard" well; both have solid table/form components which you'll want for report-heavy screens
- `react-table` / `@tanstack/react-table` — if you need advanced sortable/filterable tables beyond what the UI library gives you

Since you export to Excel/PDF from the backend, the client-side needs are light — mainly just:

- File download handling (blob response handling via axios, no special library needed)
- If you go with Pattern 1 from before (client preview before server export), you might want a lightweight print-preview library like `react-to-print` for the "quick view" — genuinely optional, only add if you build that flow

Dev/build tooling

- `eslint` + `prettier` — code quality, matters more in an offline team setting since you can't easily "just look it up" mid-review
- `typescript` — strongly recommend if not already decided; catches errors at build time instead of runtime, which is valuable when you can't quickly search Stack Overflow for a runtime error offline

For your offline bundle specifically

Since JasperReports 7.0.7 was a fairly recent release with a CVE fix (CVE-2026-6009), pin to **7.0.7 exactly**, not a range, and make sure that exact jar (plus all its transitive deps — it split into multiple optional jars in v7, so `go-offline` needs to actually pull all the ones you use: PDF exporter, XLSX exporter, fonts) ends up in your mirrored `.m2`. Test the full pipeline (JasperReports 7.0.7 + JDK 25 + Spring Boot) end-to-end before you seal the offline bundle — that's the one combination in this whole list actually worth burning an hour on now rather than discovering it broken with no internet later.

Want me to draft the actual `pom.xml` and `package.json` with all of this pinned to specific version numbers?

Yes,

Created 2 files, ran a command, read a file

A few notes on what's in these:

`pom.xml`

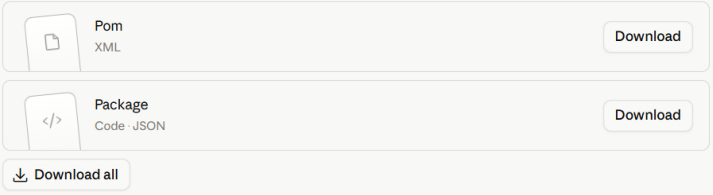
- `<maven.compiler.release>25</maven.compiler.release>` is set as a property, not hardcoded in the plugin — so if your JasperReports test on JDK 25 hits trouble, you only need to change one line (and swap `JAVA_HOME`) to fall back to JDK 21.
- Includes the official `jasperreports-maven-plugin` wired to `generate-resources` phase, pointing at `src/main/resources/reports` — adjust that path to wherever you'll keep your `.jrxml` templates. This precompiles them at build time so failures show up during `mvn package`, not at runtime in front of a user.
- Flyway included for migrations, matching the `db/migrations/` folder from the structure earlier.

`package.json`

- Went with Vite + Ant Design — Ant Design's Table/Form components will save you a lot of hand-rolling for report/dashboard screens, and Vite's dependency tree is lighter than CRA's, which matters for your offline `node_modules` transfer.
- TypeScript included by default — worth keeping for a system that needs to be audit-stable long-term, but tell me if you'd rather I strip it out for plain JS.

Next step before you build the offline bundle: run `mvn dependency:go-offline` and `npm ci` against these exact files while you still have internet, so your `.m2` and `node_modules` snapshots match precisely what's declared here — a mismatch between the pinned versions here and what actually got downloaded is the most common way these bundles go stale.

Want me to also draft a starter `application.yml` (DB connection, JasperReports config, file storage path) and a sample Jasper-backed export controller so you've got a working skeleton to validate the whole pipeline against?



Using these dependencies and stacks give one login page code both frontend and backend, include authentication and authorization and also setup swagger. And give a detailed step by step guide to setup all the tools.

Ran 3 commands

Now the backend files.

Created 8 files

- Updated pom.xml with JWT and Swagger/OpenAPI dependencies added
- Main Spring Boot application class
- Role enum for authorization
- User JPA entity
- Spring Data JPA repository for User
- Login request DTO with validation
- Login response DTO
- Generic error response DTO for auth failures

Done

Now the security layer — JWT service, filter, user details service:

Created 3 files, edited a file

- JWT generation and validation service
- UserDetailsService implementation loading users from the DB
- JWT authentication filter that runs once per request
- Remove unused import `JwtAuthenticationFilter.java` +2 -3
- Done

Now the security config, OpenAPI config, controllers, exception handler, and seeder:

Created 2 files

- Spring Security configuration - stateless JWT auth, method security enabled
- OpenAPI/Swagger configuration with JWT bearer auth scheme
- Done